

Xander Backus 6.S985 HW4 Reading assignment

Xander Backus

April 2026

1 Question 1

GRPO eliminates PPO’s learned value function (critic) by sampling G completions per prompt and using the group’s mean reward as the baseline: $\hat{A}_i = (r_i - \text{mean}(r)) / (\text{std}(r) + \epsilon)$. Where PPO uses GAE with a trained value model to estimate expected reward at each token, GRPO simply compares each completion against its siblings from the same prompt. This cuts memory substantially (no critic to store/train) and avoids the difficulty of learning accurate per-token value estimates when reward is only given at the sequence level. The trade-offs are that you must generate G completions per prompt (increasing inference cost), the baseline quality depends on group size (too small = noisy, too large = expensive), and under outcome supervision the advantage is uniform across all tokens, so unlike GAE there’s no token-level credit assignment telling the model which reasoning steps were good or bad.

2 Question 2

Rule-based rewards (e.g., regex-matching the answer against ground truth) are deterministic, cheap, and transparent, but limited to tasks where correctness is mechanically verifiable. Learned reward models generalize to open-ended tasks with subjective quality, but are approximations that can be exploited. Reward hacking manifests differently in each: with rule-based rewards, the model may game the format (e.g., always outputting "Answer:" with a guess rather than reasoning), while with learned reward models the policy can find adversarial outputs that score highly but are objectively bad, such as confident-sounding but incorrect responses the reward model hasn’t learned to penalize. This is why DeepSeekMath used iterative RL to periodically retrain the reward model on the current policy’s outputs.

3 Question 3

In SFT, the training signal is teacher-forcing: there’s one gold-standard output per input, and the model maximizes the probability of each ground-truth token

with a uniform gradient coefficient of 1. In GRPO, the model generates its own responses, receives scalar rewards, and the advantage function assigns variable gradient coefficients, positive for above-average completions and negative for below-average ones. SFT is preferred when you have high-quality paired data, need fast convergence, or are early in the training pipeline. GRPO is preferred when correct outputs are hard to obtain but easy to verify, when you want the model to discover its own reasoning strategies, or when you need to optimize composite objectives (format + accuracy jointly). The DeepSeekMath paper showed RL improves Maj@K but not Pass@K, that is to say it makes the output distribution more robust rather than expanding fundamental capabilities.

4 Secret word and screenshot

The secret phrase is "GRPO is All You Need"



```

[ ] 0 import sys, torch

print("PyTorch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
print("CUDA device count:", torch.cuda.device_count())
if torch.cuda.is_available():
    print("GPU name:", torch.cuda.get_device_name(0))

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
t = torch.randn(2, 3, device=device)
KEY = 73
cipher_bytes = [14, 27, 25, 6, 105, 32, 58, 105, 8, 37, 37, 105, 16, 38, 60, 105, 7, 44, 44, 45]

if t.is_cuda:
    cipher = torch.tensor(cipher_bytes, dtype=torch.uint8, device=device)
    decoded = cipher ^ KEY
    secret_word = "".join(chr(c) for c in decoded.cpu().tolist())
    print(f"\nGPU check passed! Secret word: {secret_word}")
else:
    print("\nNo GPU detected. Please switch to an A100 runtime.")

```

```

...
68.7/60.7 MB 49.3 MB/s eta 0:00:00
697.4/697.4 kB 59.0 MB/s eta 0:00:00
527.0/527.0 kB 46.7 MB/s eta 0:00:00
47.6/47.6 MB 54.1 MB/s eta 0:00:00
36.3/36.3 MB 64.3 MB/s eta 0:00:00

PyTorch version: 2.10.0+cu128
CUDA available: True
CUDA device count: 1
GPU name: NVIDIA A100-SXM4-40GB

GPU check passed! Secret word: GRPO is All You Need
```

Secret word: GRPO is All You Need